

DOCS_CHAIN

Docs Chain integration (§11.4.106)

Revision: 1 **Last modified:** 2026-06-11T22:40:00Z

This project wires the `vasic-digital/docs_chain` submodule (the §11.4.106 bidirectional document/DB-sync engine) to keep its tracked-doc exports (`.md` → `.html` → `.pdf`) mechanically in sync. The engine is inherited **by reference** from the `docs_chain/` submodule — never copied.

The engine's real CLI

Build:

```
cd docs_chain && go build -o /tmp/docs_chain ./cmd/docs_chain
```

```
/tmp/docs_chain version →
```

```
docs_chain (Phase 4 CLI) - sync | verify | doctor | graph | watch
```

Subcommands (from `docs_chain --help`):

```
docs_chain doctor      [--all | <context>] [--root DIR]    validate
contexts (no writes)
docs_chain sync        [--all | <context>] [--root DIR]    propagate
atomically, update state
docs_chain rebaseline  [--all | <context>] [--root DIR]    re-
baseline sync edges from AUTHORITY side only
docs_chain verify      [--all | <context>] [--root DIR]    read-only
drift check (CI gate)
docs_chain graph       <context>                          [--root DIR]    print
topo order + edges (debug)
docs_chain watch       [--all | <context>] [--root DIR]    [--debounce
300ms] sync on source change
```

Exit codes: 0 ok · 1 error · 2 conflict · 3 transform-fail · 4 cycle/config-error.
verify exits non-zero (1) on any stale node — the deterministic CI/pre-build gate.
Per-run evidence is written under `qa-results/docs_chain/<run-id>/`.

Flag-order gotcha: the CLI uses Go's flag package, which stops parsing at the first positional. Put `--root` **before** the context name: `docs_chain doctor --root . tracked_docs` (NOT `docs_chain doctor tracked_docs --root .`, which errors with "specify exactly one or --all").

The context file

Contexts live in `<root>/docs_chain/contexts/*.yaml`; state in `<root>/docs_chain/state.json`. This project's context registers docs/CONTINUATION.md and README.md, each chained `md → html → pdf`:

`docs_chain/contexts/tracked_docs.yaml`:

```
context: tracked_docs
description: helix_translate tracked-doc exports – CONTINUATION +
            README kept in sync (md -> html -> pdf) per §11.4.106
nodes:
  continuation_md: { kind: markdown, path: docs/
                    CONTINUATION.md }
  continuation_html: { kind: html,      path: docs/
                     CONTINUATION.html }
  continuation_pdf: { kind: pdf,        path: docs/
                    CONTINUATION.pdf }
  readme_md:        { kind: markdown, path: README.md }
  readme_html:      { kind: html,      path: README.html }
  readme_pdf:       { kind: pdf,       path: README.pdf }
edges:
  - { type: derive-from, from: continuation_md, to:
    continuation_html, transform: md2html }
  - { type: derive-from, from: continuation_html, to:
    continuation_pdf, transform: html2pdf }
  - { type: derive-from, from: readme_md, to:
    readme_html, transform: md2html }
  - { type: derive-from, from: readme_html, to:
    readme_pdf, transform: html2pdf }
transforms:
  md2html: { builtin: pandoc-html }
  html2pdf: { builtin: weasyprint-pdf }
```

Schema (as enforced by internal/config): top-level context (name), description, nodes (`id → {kind, path}`, `kind ∈ markdown|html|pdf|docx`), edges (list of `{type: derive-from, from, to, transform}`), transforms (`name → {builtin: pandoc-html | weasyprint-pdf | pandoc-docx}`; `exec`: for external transforms). This schema is the real one used by the engine's own dog-food context `docs_chain/docs_chain/contexts/self-docs.yaml`.

How to run

```
# build once
cd docs_chain && go build -o /tmp/docs_chain ./cmd/docs_chain &&
cd ..

# validate the context (no writes)
/tmp/docs_chain doctor --root . tracked_docs

# CI / pre-build drift gate (read-only, exit 1 if any export
stale)
/tmp/docs_chain verify --root . tracked_docs
```

```
# propagate md -> html -> pdf atomically, update state, write
  evidence
/tmp/docs_chain sync    --root . tracked_docs
```

Captured run output (real, this session)

doctor (exit 0):

```
context "tracked_docs" (.docs_chain/contexts/tracked_docs.yaml)
  parse + graph: OK (6 nodes, 4 edges)
  transforms: OK (all required tools present)
```

verify BEFORE sync (exit 1 — exports did not exist yet):

```
tracked_docs          STALE: [continuation_html
continuation_pdf readme_html readme_pdf]
```

sync (exit 0 — real pandoc + weasyprint transforms):

```
tracked_docs          applied: committed [continuation_html
continuation_pdf readme_html readme_pdf]
evidence: qa-results/docs_chain/20260611T223707Z
```

Produced files (real documents):

docs/CONTINUATION.html	9958 bytes	HTML document, UTF-8
docs/CONTINUATION.pdf	34427 bytes	PDF document, version 1.7
README.html	44918 bytes	HTML document, UTF-8
README.pdf	1247882 bytes	PDF document, version 1.7

verify AFTER sync (exit 0):

```
tracked_docs          in-sync
```

Status

Operational. Both required tools are present on this host — pandoc 3.9.0.2 and WeasyPrint 66.0 — so the full md → html → pdf sync runs end-to-end and produces real HTML/PDF documents (verified above: STALE → sync → in-sync round-trip, exit codes 1 → 0 → 0). If either tool were absent, the engine surfaces an honest tool-absent SKIP / ToolAbsentError per §11.4.106 (doctor reports transforms: WARN ... runs will SKIP-with-reason; sync rolls back with no live changes) — never a faked PASS.

Note: a sync run writes the derived .html/.pdf exports into docs/ and the repo root, and a run-evidence log into qa-results/docs_chain/. The .md authority files are read-only; only derived exports are written.